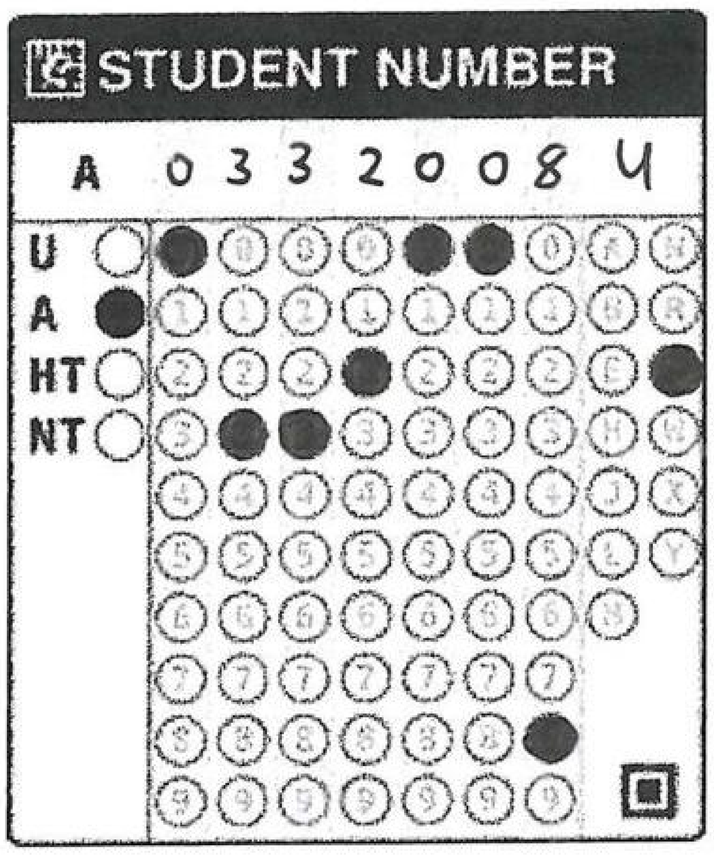
The Answer Sheet for Semester 2 AY2025/26

Write your Student Number in the box below using **(2B) pencil. Do NOT write your name.**



Section	Maximum Marks	Your Marks	Grading Remarks
Total	~~100~~ 50		

2 / 2

Box A.1. Feedback about IT5003 S2 AY2025/26:

Both feedback read, thx (2)

What is your opinion about the optional LeetCode exercises? Do you attempt them?

Great as LeetCode ~~are~~ problems are closer to technical interview questions.
I attempt all of them and ~~so~~ I also try extra ones when I have time. I ~~am~~ am active on Discord ~~to~~ to share my solutions.

What is your opinion about this new 'interesting' 2nd IT5003 midterm format?

Very nice ~~[illegible]~~ as it tests not only ~~[illegible]~~ our skill of analysing a problem/writing ~~a~~ the code but also reading others' code (and correcting them) which ~~is~~ is a very practical skill, in group work for example.

16 / 16

Good Job, O(n) solution (small issue forgiven - but see annotation if any) (16)

Box B.1. Sort Bit String

```
class Solution:
    def sortBinary(self, s: str) -> int:
        s = list(s) # convert to list of chars
        ans, n = 0, len(s)
        for i in range(n):
            for j in range(n-1-i):
                if s[j] > s[j+1]:
                    s[j], s[j+1] = s[j+1], s[j]
                    ans += 1
    return ans
```

My analysis of the generated code above is:

TLE The answer is correct, but it ~~uses~~ is based on bubble sort and ~~could be~~ has $O(n^2)$ time complexity in the worst case, which is too slow.

This problem is essentially an inversion index problem. For general lists we have an $O(n \log n)$ solution based on merge sort, but here as s is composed of 0s and 1s only, we can simply count based on the 1s.

Basically, we reverse the list first; then convert it into a list of integers; for every 1 we encountered, add ans by i-j where i is the current index and j is the number of 1s up to now. This is the number of 'steps' it is required to 'travel'. Return ans at the end.

The Python code is shown below:

```
def sortBinary_new(self, s: str) -> int:
    s = reversed(list(s))      # reverse the list
    s = map(int, s)            # convert '0' to 0 and '1' to 1
    ans = i = j = 0            # initialise
    for c in s:                # traverse the list
        if c:                  # if the character is 1
            ans += i - j       # update the answer
            j += 1             # the number of 1s so far increases by 1
        i += 1                 # the index increases by 1
return ans                     # this is O(n) as we traverse once only
```

That solution runs in $O($ n $)$.

10 / 16

O(n) (16)

missing WA analysis:
head not updated (-1)

missing WA analysis:
rightmost duplicate
not deleted (-1)

delete head still WA /
RTE (-4)

Bx 3.2. Simple Linked List Task

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def deleteDuplicates(self, head: ListNode) -> ListNode:
        prev = None
        curr = head
        while curr:
            count = 0
            checker = head
            while checker:
                if checker.val == curr.val:
                    count += 1
                checker = checker.next
            if count > 1:
                prev.next = curr.next
            else:
                prev = curr
            curr = curr.next
        return head
```

The logic is mostly correct, but prev is initially set to None and calling prev.next can cause the code to crash, when the first node is a duplicate (count > 1), thus RTE.

My analysis of the generated code above is:

RTE and TLE ~~It does not remove the last occurrence of any duplicate element, as at that point count == 1 (checker only traverses to the right and never goes back).~~ Also it can be $O(n^2)$ in the worst case.

A better ($O(n)$) solution: maintain a list of length 100 (as all values are between 1 and 99) that records the number of occurrences ~~each~~ of the value of each node. Traverse the SLL to ~~initialise~~ populate this list. Traverse the SLL again to remove all duplicates as needed.

Pseudocode: (as Python code is too long and there isn't enough space)

```
a = [0] * 100
for every node in the SLL:
    a[node.val] += 1
for every node in the SLL:
    if a[node.val] > 1:
        remove the node
# Thus this is O(n)
```

That solution runs in $O(\underline{\quad n \quad})$.

O(n log n), fastest (16)

Box B.3. Minimum Number of Moves

```
class Solution:
    def minMoves(self, nums, t):
        moves = 0
        while True:
            all_good = True
            for value in nums:
                if value < t:
                    all_good = False
                    break
            if all_good:
                return moves
            nums.sort(reverse = True)
            x, y = nums[0], nums[1]
            nums.pop(0)
            nums.pop(0)
            new_value = 4 * x + 2 * y
            nums.insert(0, new_value)
            moves += 1
    return moves
```

as we need the smallest elements

My analysis of the generated code above is:

WA and TLE Should have been reverse = False (ascending), but even after this it is still too slow as the list is sorted every pass. We can use a min heap to improve to O(n log n).

```
def minMoves_new(self, nums, t):
    from heapq import heapify, heappop, heappush
    ~~nums =~~ heapify(nums)
    moves = 0
    while True:
        x = heappop(nums)
        if x ≥ t: return moves        # that's when all elements are ≥ t
        y = heappop(nums)
        heappush(nums, 4 * x + 2 * y)
        moves += 1
```

Each pass inside the loop takes only O(log n) due to property of min heap and the worst case takes O(n) passes. Thus O(n log n).

(Also the loop is guaranteed to end as per the problem description.)

That solution runs in $O($ n log n $)$.

– END OF PAPER; All the Best –